

Course Overview

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

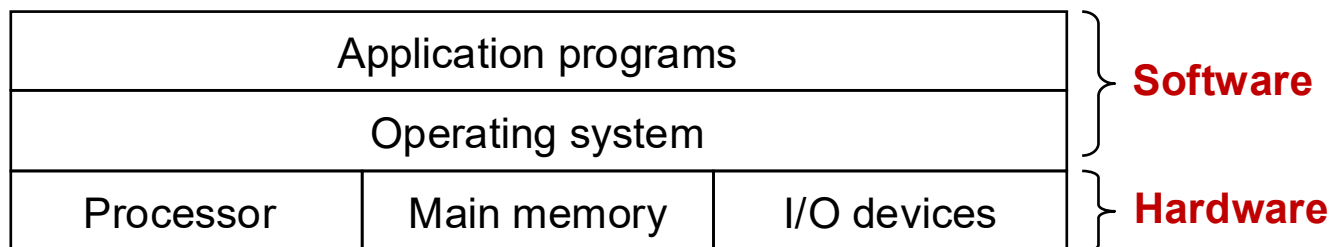
Overview

- **Course theme**
- **Main topics**
- **Logistics**

Course theme

■ Exploration of operating system concepts and concurrent programming

- Know and understand the purpose of an operating system and the services provided to users, processes, threads, and other systems
- Learn to design and implement effective concurrent programs, addressing challenges like synchronization, resource sharing, and deadlock prevention



Course theme

- Explore foundational concepts that are shared across all major operating systems
- While examples from other OSes will be highlighted, practical work will **exclusively focus on Linux** due to its accessibility and versatility for experimentation
 - Leverage your VM from ARQCP or any other Linux distribution to apply concepts effectively
- **Required skills**
 - Solid understanding of computer architecture principles
 - Proficiency in C programming
 - Familiarity with basic Linux commands and operations

Why SCOMP?

■ An exciting time for OS designs

- The emergence of new hardware, smart devices, self-driving cars, and advanced data centers is revolutionizing OS design
- Current challenges include improving performance, battery efficiency, security, and system isolation

■ Foundational principles for systems in general

- Master key principles such as caching, concurrency, memory and I/O management, protection, and security

Why SCOMP?

■ Complexity in software systems

- OSes illustrate the evolution of complex systems
- Many of you will contribute to large-scale software projects, making this understanding invaluable

■ Understand what you use

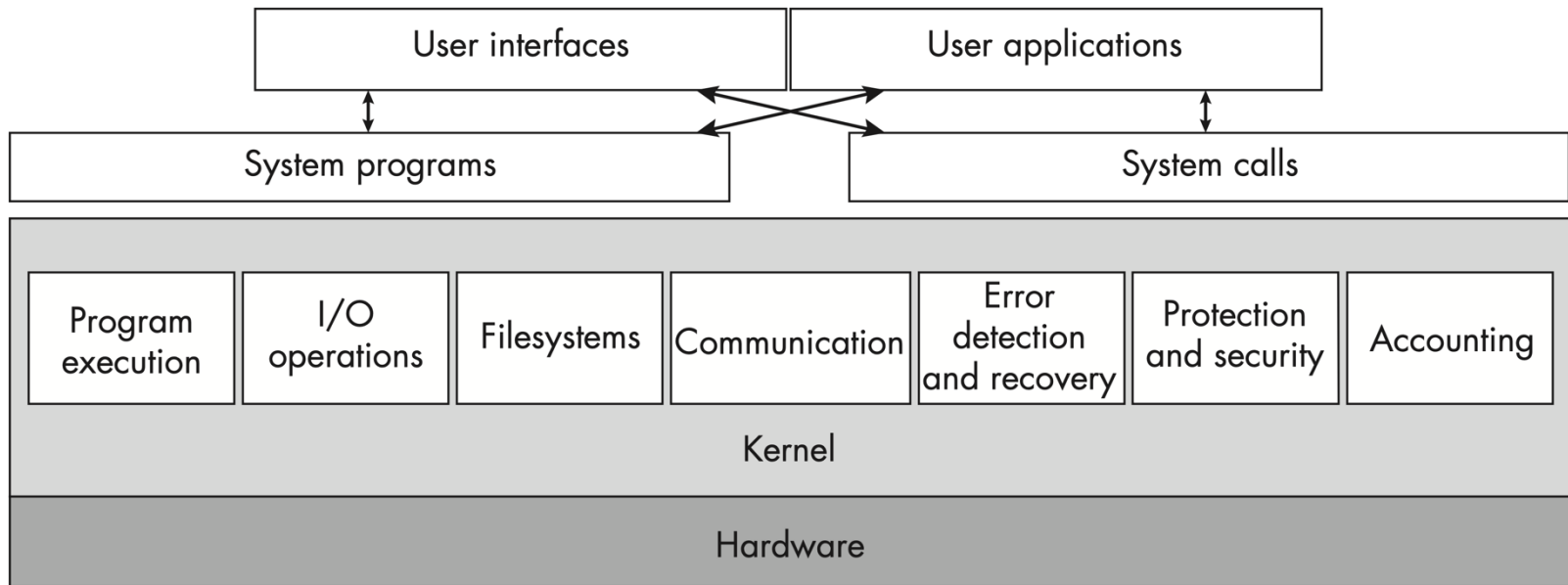
- System software often feels like a black box
- Gaining insights into OS design enhances your effectiveness as a programmer

Overview

- Course theme
- **Main topics**
- Logistics

Operating system

- A layer between applications and hardware that provides a set of services and abstractions
 - All the code that you didn't have to write to implement your application



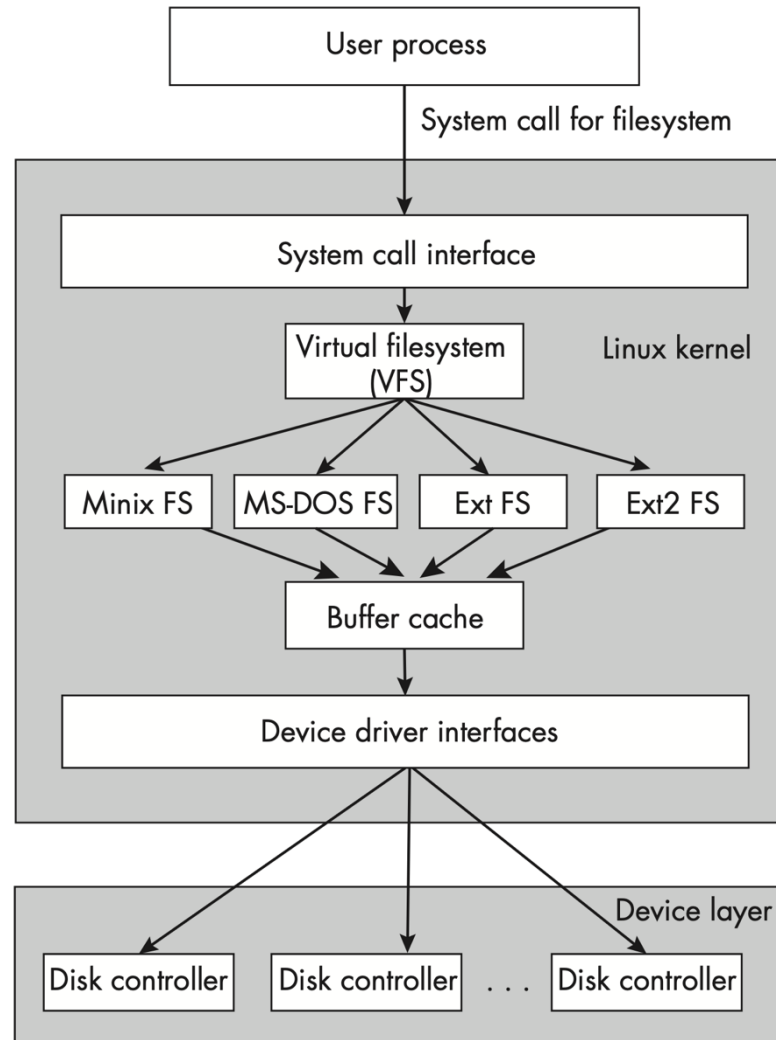
Example: Filesystems

- One of the purposes of an OS is to **hide the peculiarities** of specific hardware devices from applications
- Many kinds of storage media used in a typical computer
 - Hard disks have varying interfaces
 - Different size HDDs must be accessed in different ways
 - Storage devices might also have different formats
- Different storage technologies require different kinds of maintenance
 - Magnetic disks are sensitive to fragmentation
 - To minimize performance and wear issues, the filesystem must interact with SSDs differently than with magnetic disks

Filesystems: Standardized interface

- **UNIX-like OSes provide a simple mechanism for interacting with storage devices in the computer**
 - `open()`, `close()`, `read()`, `write()`, etc.
- **A Virtual File System (VFS) presents a single unified view of all devices and files in the computer**
 - Root of the virtual filesystem is “/”
 - Storage devices are mounted at specific paths, e.g., “/mnt/cdrom”
 - Every file can be accessed by a path from the root of the filesystem
- **The VFS creates a common interface for system calls needing access to files and the filesystem, regardless of their underlying types**

Virtual File System



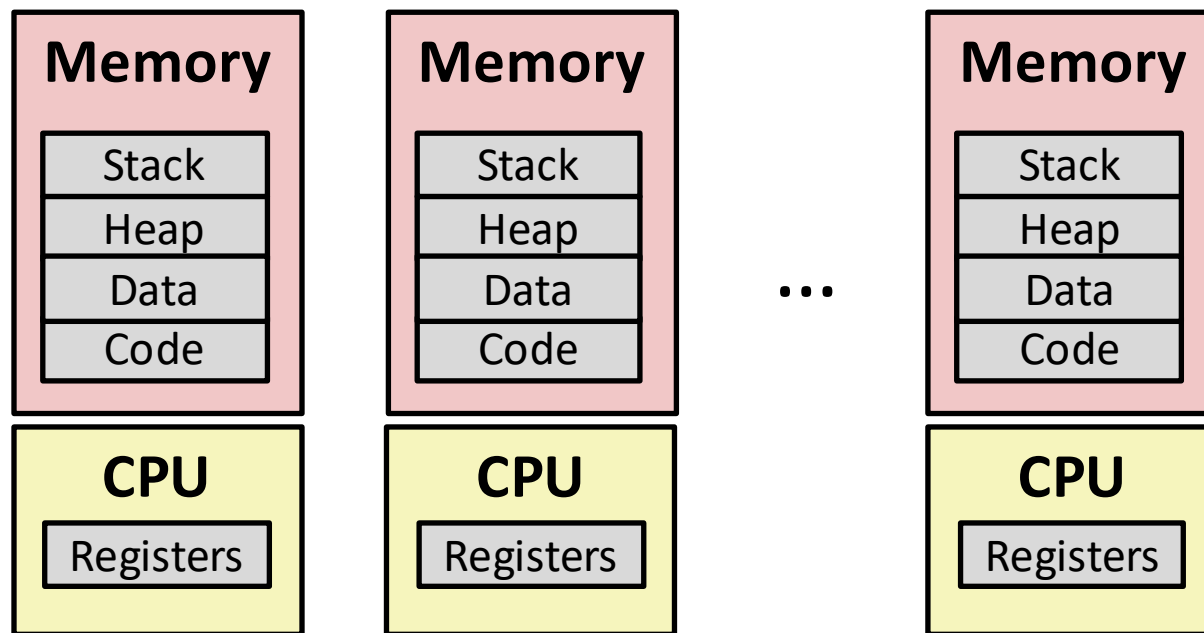
Filesystems: Standardized interface

- **In fact, other devices use essentially the same interface!**
 - Console input and output (`printf / scanf` use `read/write`)
 - Socket communications
 - Pipes between processes
 - ...
- **Only real API difference is how to open each device**
- **See the document “System-level I/O” available in Moodle**

Multiprocessing

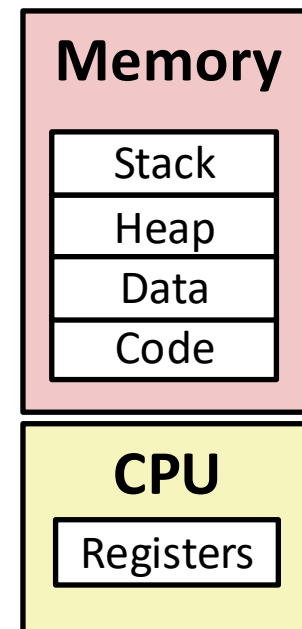
■ How can many programs share a computer efficiently?

- **Scheduling**: (fair) sharing of the CPU, prevent monopolization of the CPU
- **Virtual memory**: protect process's memory from one another



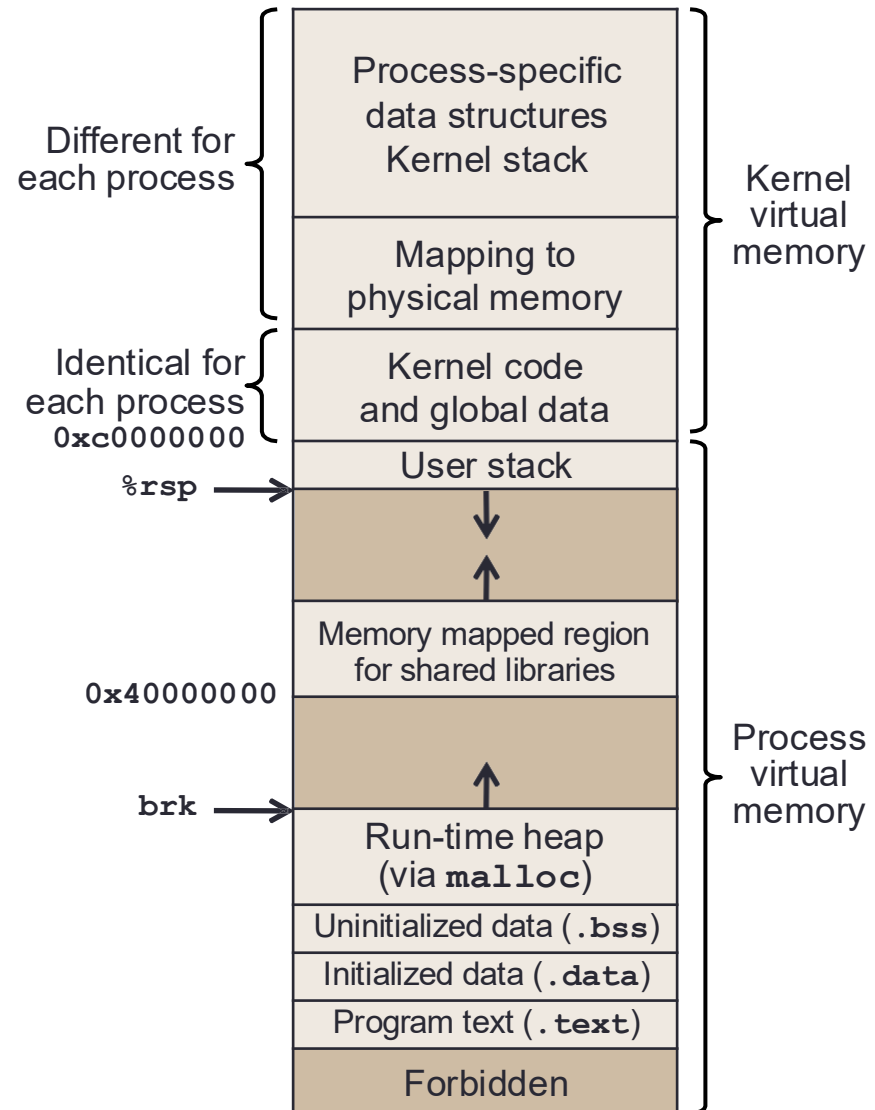
The process abstraction

- A process provides each program with two key abstractions
 - A **private address space** and a **private view of the CPU**
- **Private address space**
 - Each program seems to have exclusive use of main memory
 - Provided by a kernel mechanism called **virtual memory**
- **Logical control flow**
 - Each program seems to have exclusive use of the CPU
 - Provided by a kernel mechanism called **scheduling**



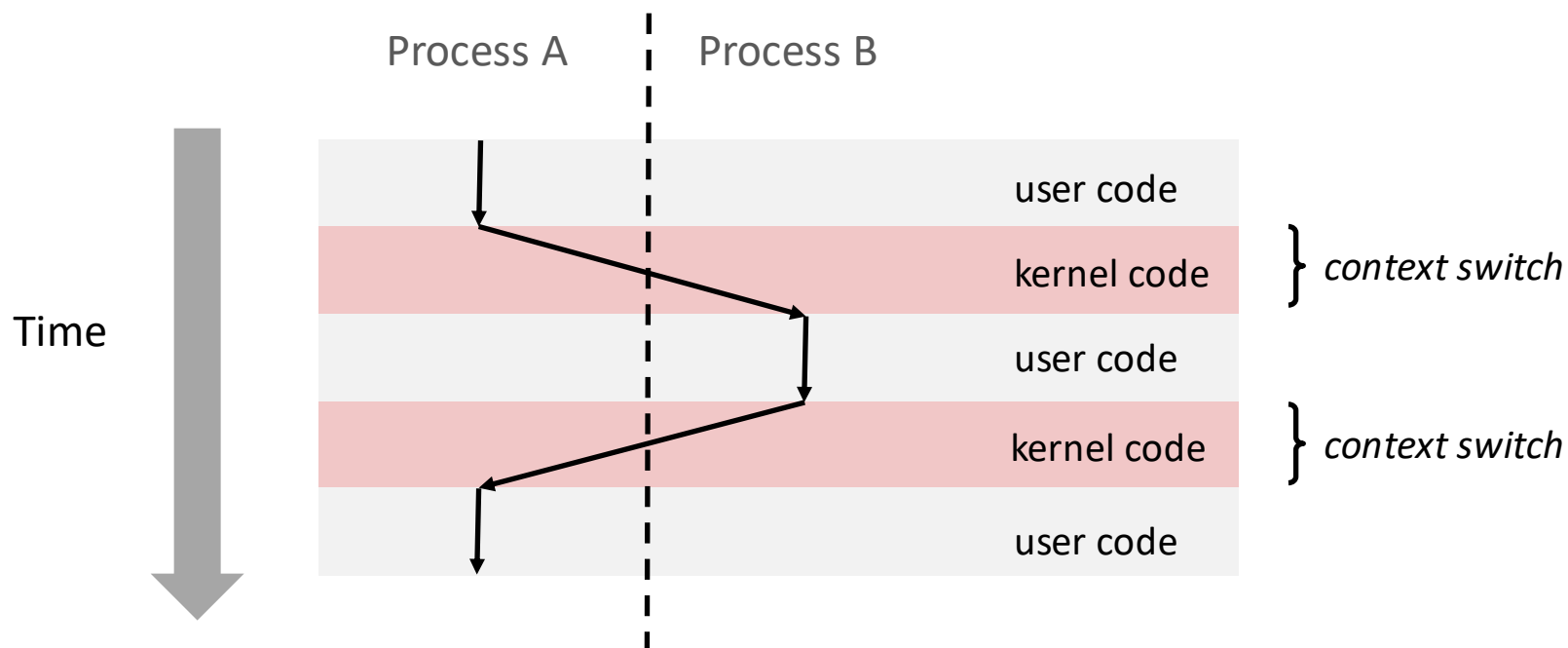
Virtual memory

- Each process has its own isolated **virtual address space**
 - Processes can view memory as a simple linear array
- Processes have identical memory layouts
 - Simplifying compilation, linking and loading



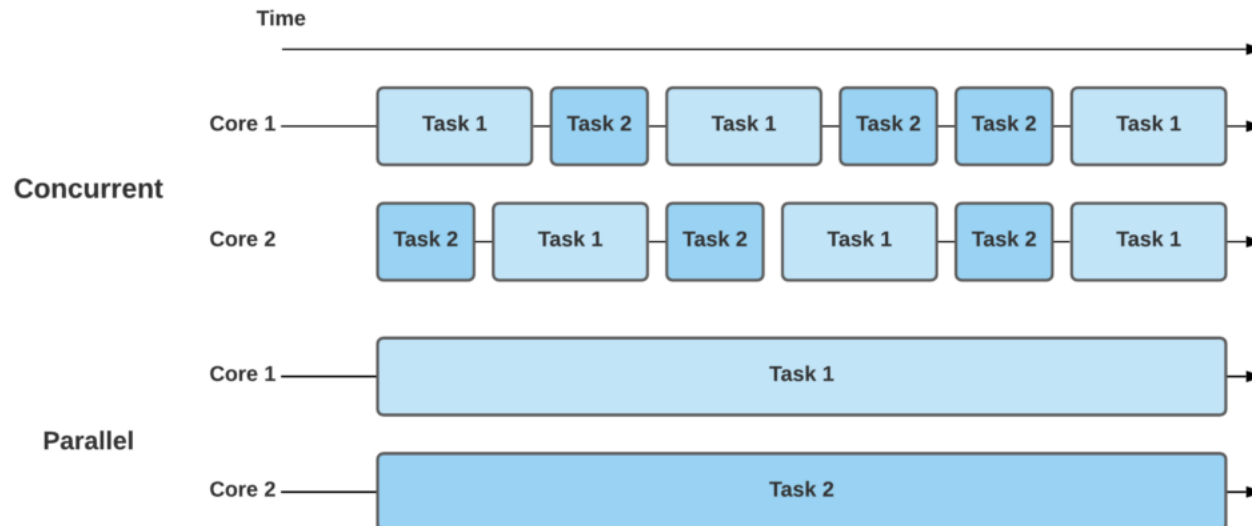
Scheduling

- Scheduling is a crucial feature of an OSs that govern sharing CPU time among the numerous running tasks
 - Control flow passes from one process to another via a **context switch**



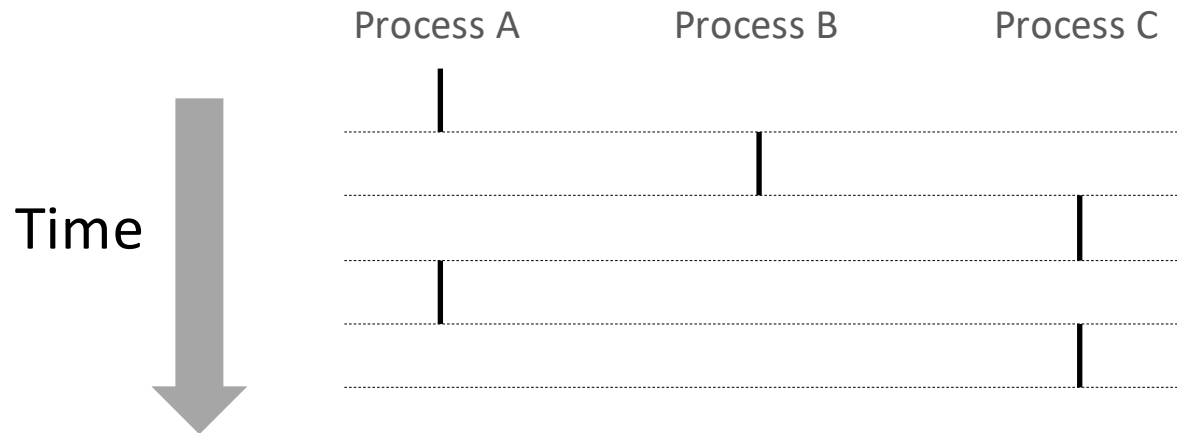
Concurrent programming

- **Concurrency** means that multiple tasks can be executed in an overlapping time period
- **Parallelism** is the ability to execute independent tasks of a program in the same instant of time



Concurrent programming is hard!

- **Thinking about all possible sequences of events is at least error prone and frequently impossible**
 - The human mind tends to be sequential
 - The notion of time is often misleading



Process execution

```
int main(int argc, char** argv){
    pid_t pid;
    int x = 1;

    pid = fork();

    if(pid == 0){
        /* Child */
        printf("child : x=%d\n", ++x);
    }else{
        /* Parent */
        printf("parent: x=%d\n", --x);
    }
    return 0;
}
```

ex.c

- Can't predict execution order of parent and child
- Duplicate but separate address space
 - x has a value of 1 when `fork()` returns in parent and child
 - Subsequent changes to x are independent
- Shared open files
 - `stdout` is the same in both parent and child

```
linux> ./ex
parent: x=0
child : x=2
```

```
linux> ./ex
child : x=2
parent: x=0
```

```
linux> ./ex
parent: x=0
child : x=2
```

```
linux> ./ex
parent: x=0
child : x=2
```

Why study concurrency?

■ Improved performance

- Concurrency allows multiple tasks to run simultaneously, making the most of multicore processors, enhancing overall system performance

■ Enhanced responsiveness

- Concurrent systems remain responsive even when some tasks are blocked or busy, providing a smoother user experience

■ Resource utilization

- Concurrency optimizes resource utilization, ensuring that CPU, memory, and other resources are efficiently used

■ Scalability

- Concurrency supports the scaling of applications to accommodate growing workloads without significant performance degradation

Concurrent programming is hard!

- **Several applications run as a collection of multiple processes that work together, rather than as a single process**
 - Some are even more complex, running as several multithreaded processes
- **Processes that work together need to coordinate their actions by communicating with each other**
 - **Inter-process communication** (IPC) refers to the exchange of information between processes
- **When processes exchange data and/or share resources, **synchronization** is required**
 - To ensure consistency, avoid race conditions, and preserve correct execution order

Concurrent programming is hard!

■ Classical problems of concurrent programs:

- **Race conditions** – outcome depends on arbitrary scheduling decisions elsewhere in the system
- **Deadlock** – improper resource allocation prevents forward progress
- **Livelock/starvation/fairness** – external events and/or system scheduling decisions can prevent sub-task progress

■ Many aspects of concurrent programming are beyond the scope of our course, but not all...

- We'll cover some of these aspects in the next lectures

Concurrency is hard to test and debug

- If I haven't persuaded you that concurrency is tricky, here's the worst of it: it's very hard to test and debug
- **Concurrency bugs exhibit very poor reproducibility**
 - These kinds of bugs are called **heisenbugs**, which are nondeterministic and hard to reproduce
 - A heisenbug may even disappear when you try to look at it with `printf` or a debugger!
 - As opposed to a **bohrbug**, which shows up repeatedly whenever you look at it. Almost all bugs in sequential programming are bohrbugs
- **Part of the point of this is to scare you a bit**
 - Over the next several weeks, we'll see the main ways to design concurrent programs so that they are safer from these kinds of bugs

Overview

- Course theme
- Main topics
- **Logistics**

Textbooks

■ **Operating System Concepts, 10th edition**

- Authors: Silberschatz, Galvin, Gagne. Publisher: Wiley
- High-level coverage of major themes across many OSes

■ **The Linux Programming Interface: A Linux and UNIX System Programming Handbook**

- Author: Michael Kerrisk. Publisher: No Starch Press
- The Linux programming interface for concurrent programming

■ **System Programming in Linux: A Hands-On Introduction**

- Author: Stewart N. Weiss
- Programming proficiency is forged through active practice

Lectures

- **Operating systems: general principles and components**
- **The process/thread abstractions**
- **Process/thread communication**
- **Synchronization on shared data structures**
- **Process/thread (real-time) scheduling**

Practical and Lab classes

- **Process management**
- **Inter-process communication**
- **Synchronization on shared data structures**
- **Thread management**

Getting Help

- **Course Web Page: <http://moodle.isep.ipp.pt>**
 - Copies of lectures, assignments, exams, solutions
 - Message board

- **Staff mailing list**
 - Send email to individual instructors to schedule appointments or getting help

Lab rationale

- **Each module (two weeks) has a well-defined goal**
 - Doing the module's exercises should result in new skills and concepts
 - Work **individually**

- **Group project (coming from LAPR4)**
 - Evaluates acquired skills in previous modules

Grading

- **Lab classes (30%): minimal grade of 8.5**

- Group project

- **Final exam (70%): minimal grade of 8.5**

- Can only attend if Lab's grade is ≥ 8.5
- **Closed-book exam**, with theoretical questions, code interpretation and code development exercises

- **Supplemental exam**

- Retake the final exam only **or** final exam and additional closed-book exercises to replace lab's grade

*Welcome
and Enjoy!*